

APPLICATION DOSSIER

From the customer experience to the backend.

Product, architecture, administration, integrations and operating guide

Prepared 28 September 2026
Project version 0.1.0 · Local implementation snapshot

77

public menu entries

31

database tables

53 / 53

automated tests passed

Prepared from the application source, a read-only database inspection, project documentation and local verification. This is a description of the current implementation, not a declaration of completed production deployment. Passwords, tokens and customer records are excluded.

Reading this report

NVO combines a bilingual restaurant website with a staff workspace for menu management, order requests, coupons, content and social publishing. The aim is to help customers discover food, take action through WhatsApp, and return through offers and useful restaurant updates.

Status language

Label	Meaning
Implemented	Present in the inspected source and local application. This does not by itself prove live provider operation.
Configured locally	A value exists in the current restaurant database or workspace.
Awaiting activation	Code exists, but hosting, account authorization, credentials or live verification remain.
Outstanding	A requested capability is incomplete or absent from the current implementation.

Scope and evidence

- Fresh verification on 28 September 2026: npm test completed with 53 passes and zero failures. Tests use isolated data and simulated publishing providers.
- Earlier browser, accessibility and Lighthouse reports are stored under artifacts/. Their historical results are identified separately; the browser suite was not rerun for this document.
- The report covers the implemented product and relevant gaps from the original brief. It does not reproduce the entire source code, credentials, or personal customer data.
- Current business settings and record totals are a snapshot; staff changes after generation will not update this PDF automatically.

Document map

Business profile and status → architecture and frontend → orders and coupons → admin and integrations → security, operations and launch gaps → detailed API, schema, environment and source references.

Source: README.md; instructions.txt; research/README.md; tests/*.test.ts

Restaurant profile and current data

Field	Current local value
Public name	NVO Restaurant
Location	Commissariat, Agblangandan, Cotonou, Bénin
WhatsApp order destination	+22950924184
Currency	XOF / FCFA
Map coordinates	6.3729882, 2.4852719
Map link	https://www.google.com/maps?cid=5172180877275416931
Instagram profile	https://www.instagram.com/nvo_benin/
Business email	Not yet provided
Opening hours	Not yet provided
Public price setting	Enabled; individual meals can override
Show sample editorial content	Disabled
Referral configuration	Enabled; 5-minute claim window
Stored social accounts	0

The location and WhatsApp number were supplied by the project owner. The map entrance, live WhatsApp routing, account ownership, final opening hours and policies still need restaurant confirmation. The Instagram URL is a website link, not evidence that publishing authorization is connected.

Brand direction: royal blue, white and gold; seafood and Nigerian cuisine. The selected locality is Agblangandan, Cotonou, Bénin. Earlier printed materials contain a different address; the current owner-selected location takes precedence.

Source: data/nvo.sqlite settings (read-only); lib/seed.ts; research/restaurant-research.md; research/current-menu-and-seo.md

Feature inventory and delivery status

Area	Current implementation	Remaining boundary
Customer site	Home, menu, dish pages, gallery, story, contact, events, news, specials, offers, loyalty and testimonials	Final restaurant content and business policies
Ordering	Persistent cart, server quotes, saved requests, prepared WhatsApp messages	Customer presses Send; no online payment gateway
Reservations	Saved request and WhatsApp handoff; staff status changes	Staff must confirm availability
Coupons	Named cards, claim rules, wallet, expiry, counter redemption	Verified customer identity and automatic hold release
Referrals	Links, visit tracking, claim window	Automatic referrer reward on a completed sale
Staff tools	Content, photos, orders, settings, analytics, publishing desk	Staff-account management and password recovery UI
Social publishing	Connections, captions, queue, schedules, retries and receipts	Real provider authorization and live acceptance checks
Translation	Local English/French model worker	Human review of food names and offer terms
Hosting	Local custom Node server; Oracle deployment templates	Oracle access, public deployment, backups and restore test

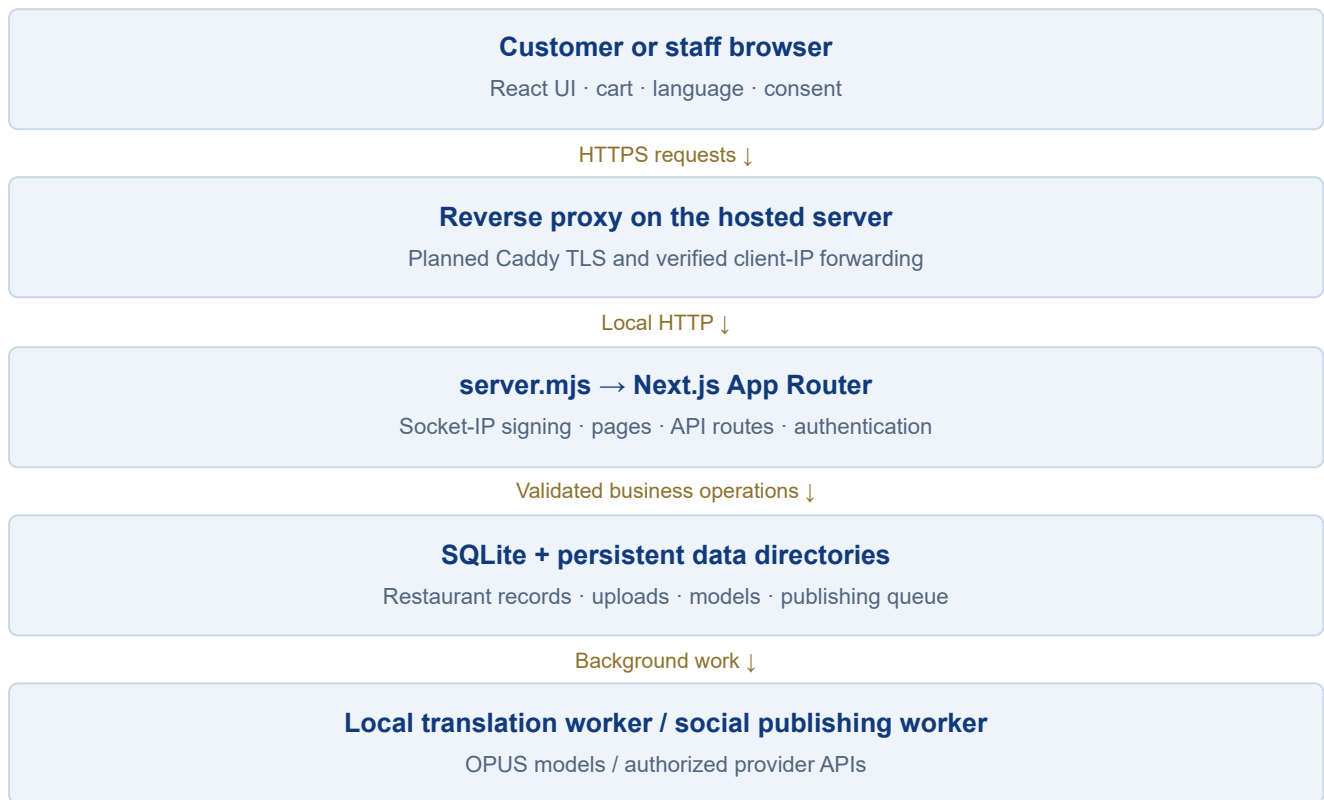
Stored records versus public content

The database holds 1 campaigns, 12 categories, 3 events, 1 loyalty, 80 meals, 4 posts, 1 specials, 1 testimonials. Those totals include drafts, archived items and sample content. The inspected catalogue rules expose 77 live meal entries across 12 live categories.

The original 63-section brief is broader than the delivered implementation. “Built locally” must not be read as “every requested feature complete” or “social posting verified in production”.

Source: README.md; lib/catalog.ts; lib/types.ts; current database entry aggregates

Architecture and request flow



The application is one deployable Node service rather than separate frontend and backend projects. Pages can obtain restaurant data on the server, while interactive client components call JSON endpoints. The root layout loads the public catalogue and initializes the client provider.

server.mjs loads environment configuration, prepares Next.js, and binds to 127.0.0.1 by default on port 3000. It signs the verified network address on each request. Production uses this entry point rather than bypassing it with a plain framework server.

SQLite and locally stored files require a durable filesystem. The current deployment design is a single server; multiple independent replicas with their own files would not share restaurant state.

Source: server.mjs; app/layout.tsx; components/provider.tsx; lib/db.ts; instrumentation.ts

Technology stack and project layout

Layer	Package / runtime	Purpose
Server runtime	Node.js >=24	Native SQLite and application process
next	16.3.6	Pages, rendering, routes, images, metadata
react	19.3.0	Interactive UI
react-dom	19.3.0	Browser rendering
zod	4.6.5	Runtime input validation
sharp	0.35.4	Social image preparation
qrcode	1.5.4	Coupon verification QR codes
@huggingface/transformers	4.3.0	Translation pipelines
onnxruntime-web	1.26.0	CPU/WASM inference

Directory / file	Responsibility
app/	Page routes, API routes, metadata and shared styles
components/	Customer pages, menu, cart UI, admin, coupons and publishing tools
lib/	Database, authentication, catalogue, validation and business logic
lib/social/	Account authorization, provider calls, captions, jobs and delivery
public/	Deployed images, fonts and videos
data/	SQLite, uploaded media, model files and generated social images
scripts/	Setup, imports, fixtures, audits and workers
tests/ and artifacts/	Automated checks and local evidence
research/ and instructions.txt	Requirements, decisions and operating notes
deploy/oracle/	Prepared service/proxy/environment templates; not deployed

Source: package.json; package-lock.json; source directory inventory

Frontend pages and navigation

Route	Customer purpose
/	Restaurant introduction, food highlights, specials and calls to action
/menu and /menu/[id]	Search/filter dishes, view details, choose quantities
/cart	Review selection, customer details, coupon, delivery/pickup/dine-in
/reservation	Request a table with date, time and party size
/specials and /specials/[id]	Discover current meals or kitchen spotlights and enquire
/offers; /promotions	Coupon experience; old promotions alias redirects to offers
/referral and /r/[code]	Create/share referral links and record invited visits
/news, /events and detail URLs	Published restaurant content and occasions
/about, /gallery, /contact	Restaurant story, photographs, film, address and map
/loyalty and /testimonials	Staff-managed recognition and guest stories
/subscribe	Consent-based WhatsApp marketing signup instructions
/privacy and /terms	Current policy copy; owner review required
/admin	Authenticated staff workspace

French public URLs use /fr; English uses the unprefixed canonical route. /en aliases redirect permanently. API and admin endpoints are not separate translated deployments. Unknown content returns a real not-found response.

The design uses responsive layouts, a mobile-friendly navigation system, persistent cart controls, a vector chef host, optional spoken welcome, motion preferences and reduced-motion support. Content and offer eligibility are enforced in the backend, not just by hiding buttons.

Source: app/[...slug]/page.tsx; app/page.tsx; proxy.ts; components/shell.tsx; components/chef.tsx; components/motion.tsx

Menu, media and bilingual content

The imported PDF menu defines 77 dishes, portions and extras in 10 categories. The current database exposes 77 live meal entries; 8 have no fixed price and 10 are marked for pre-order.

- Categories include rice, other meals, pepper soups, Nigerian soups, swallow, barbecue, shawarma, seafood, seafood boil packages and extras.
- Meal records support English/French titles and descriptions, category, photo, price visibility, availability, pre-order, featured state, badges and sorting.
- Some differently priced portions were transcribed as separate entries because the supplied PDF did not identify their exact portion sizes. Staff can clarify the labels later.
- Global and per-meal price visibility are applied by the public catalogue and quote logic. Unknown/hidden prices remain null; they are not treated as free.
- Search supports English/French content and accent-insensitive matching. Dish pages include related choices and selection controls.
- The importer previews before applying, creates a consistent SQLite backup and preserves staff edits and historical references. It does not reset the restaurant database.

Media management

Staff can upload JPEG, PNG or WebP images up to 8 MB. Server code checks basic file signatures and writes random filenames under data/uploads. Records track the original display name and public URL. Images referenced by entries, including drafts, cannot be deleted until references are replaced.

NVO photographs are supplemented by labelled illustrative images with credits in lib/menu-photos.ts. Unsuitable or unavailable dish photos use a branded placeholder. The kitchen film loads on playback; local font files include their licenses.

Upload signature checks are not a complete malware-scanning or image-content-moderation system. Avoid describing them as such.

Source: lib/current-menu.ts; lib/menu-import.ts; lib/menu-photos.ts; components/menu.tsx; app/api/[...path]/route.ts

Orders, quotes and reservations

Browse → Add to cart → Server quote → Save request → Open WhatsApp → Customer presses Send → Staff confirms

The browser stores meal identifiers and quantities. The backend reloads each meal, verifies its current availability and category, checks quantities, and calculates prices and coupon eligibility. Client-submitted totals are never the authority.

- Order validation permits 1–40 distinct lines and quantities of 1–30 per line. Customer name, phone, notes, method and delivery address are collected as applicable.
- A request key makes repeated submissions return the original order instead of silently creating another. The order and any coupon hold are committed transactionally.
- After success, only submitted quantities are removed; new items added during the request are retained. Cart changes synchronize across tabs. A same-tab confirmation can be restored for 24 hours.
- If any line price is unknown/hidden, the quote does not expose a calculable total or savings. Staff can enter a manual quote later.
- Order states are request_created, accepted, fulfilled_paid and cancelled. Only staff-marked paid/fulfilled orders count as confirmed paid outcomes.
- Reservation requests store name, phone, date, time, guests and message. Dates are evaluated in the Cotonou offset; staff can mark requested, confirmed or cancelled.

A stored order, reservation or WhatsApp click does not prove a message was sent, a table is confirmed, money was received, or food was delivered. Payment and fulfilment happen outside the website and require staff confirmation.

There is no integrated card-payment checkout, automatic WhatsApp order-message sending, inventory management, delivery tracking or automatic restaurant booking allocation in this implementation.

Source: lib/cart.ts; lib/pricing.ts; lib/validation.ts; components/pages.tsx; app/api/[...path]/route.ts

Coupons, wallet and referrals

Campaign records define percentage, fixed-amount or free-meal rewards, minimum spend, discount cap, total claim capacity, opening/closing times, activation delay and validity period. Staff can publish, hide or archive a campaign.

- Guests provide their name, claim an eligible offer and receive a unique NVO code. Issuance stores a snapshot of the terms so later campaign edits do not silently rewrite an issued reward.
- The browser-linked wallet displays reward ownership, activation and expiry. A downloadable PNG card contains the name, code, validity, conditions and a staff-verification QR code.
- Claiming is transactional. The current backend enforces one claim per browser identity and one network/IP per offer. Although the data model contains perCustomerLimit, current claim logic hard-caps browser claims at one.
- Shared Wi-Fi is treated as one network. Changing network or clearing browser state can affect recognition; this is not verified human identity or an OTP-protected customer account.
- A referral link records its owner and the invitee's first visit. The configured claim window is checked server-side; self-referral and expired-window claims are rejected. Automatic rewards to the referrer after a sale remain outstanding.

Counter redemption

Authorized staff look up or scan a code, verify the customer name, enter receipt items and prices, calculate the discount and confirm payment. The server records a counter sale and atomically marks the coupon redeemed. Pending, expired, held or already-used rewards cannot be redeemed again.

Current gaps: recurrent randomized coupon drops, cross-device wallets, verified customers, automatic referrer rewards and automated release of abandoned order-held coupons. The current implementation is a configurable campaign system, not the complete proposed growth engine.

Source: lib/rewards.ts; lib/coupons.ts; lib/network.ts; components/coupons.tsx; components/coupon-desk.tsx

Staff workspace and permissions

The admin desk groups menu and guest operations, restaurant storytelling, and tools/settings. It includes quick tasks, search/filtering, a mobile section selector, staff guidance and contextual empty/error states.

Role	Main permissions enforced in code
Owner	Restaurant settings; all content, offers, operational and publishing controls
Manager	Content and campaigns; orders/reservations; coupon redemption; connect accounts and manage subscribers
Content	Non-campaign editorial/menu content, photos, translations, captions and publishing job actions
Service	Orders, manual quotes, reservations and coupon verification/redemption
Analyst	Read staff dashboard and activity data; no content or operational writes

Content edits include title, description, language variants, image, publication state and timing. Meals have category, pricing and availability controls. Editorial entries may choose website-only publishing, configured social defaults or explicit destinations.

- Entry “delete” actions archive records to preserve historical order/coupon references, and cancel eligible unsent social jobs.
- Only the owner changes core restaurant settings. Campaign create/archive controls are limited to owner and manager.
- Owner/manager account management handles connecting, selecting defaults, disconnecting and subscriber removal. Content staff can work with permitted publishing tools but cannot connect accounts.
- The initial staff account is bootstrapped from private environment configuration when no matching account exists. Changing bootstrap credentials does not change an existing account’s password.

Role checks are implemented, but a complete UI for creating/managing staff accounts and password recovery is still outstanding. A production owner-access recovery procedure must be established.

Source: components/admin.tsx; components/social-admin.tsx; lib/auth.ts; app/api/[...path]/route.ts; app/api/admin/social/[action]/route.ts

Publish everywhere: supported channels

Destination	What the current code sends	Activation requirements
Website	Published stories, specials and events	Valid staff login and content
Facebook	Photo and caption to selected Page	Meta developer app, permissions, owner authorization
Instagram	Photo container then publication	Professional account linked to selected Facebook Page and publishing access
TikTok	Photo upload into the creator's inbox	Developer app, upload access, domain verification, per-post consent; finish in TikTok
WhatsApp	Approved marketing template to explicit subscribers	Cloud API sender, valid templates, webhook, authorized credentials and any required billing

The integration currently supports one photo per social post. Video/Reels, carousels, WhatsApp Status and WhatsApp Channel publishing are not implemented. A website Instagram link and an order-chat number are not equivalent to connected API accounts.

Staff flow

Write a story, event or special → choose photo and language → review generated captions → choose destinations → save/publish now or set a future start → review individual delivery results. Connecting a new account does not automatically backfill old posts.

Current database snapshot: 0 stored social accounts. Publishing code and mocked tests exist; real provider publishing has not been demonstrated for NVO. Provider rules and approval requirements must be rechecked during activation.

Approved account access, callback URLs, publicly accessible photos and a live HTTPS origin are essential. The application deliberately rejects localhost as a social publishing origin.

Source: [research/social-publishing.md](#); [lib/social/accounts.ts](#); [lib/social/providers.ts](#); [lib/social/security.ts](#); [components/social-admin.tsx](#)

Publishing queue and delivery reliability

Website entry changes and their publishing-queue changes are saved in one SQLite transaction. Jobs are durable records, with a unique entry/account pair, snapshotted payload, revision, next-attempt time and provider receipts.

- The embedded worker starts through instrumentation.ts and polls every 10 seconds. An external worker mode is available on the same persistent host/database; off mode is for disabled delivery/testing.
- Per-account leases prevent concurrent workers from publishing two jobs for the same account. The worker rechecks eligibility and account status before sending.
- Future content waits until its scheduled time; drafts, sample entries, archived and expired content are excluded from new sends.
- Transient/rate-limit rejections back off. Authentication failures pause affected accounts. One destination failing does not automatically stop another destination.
- Interrupted or ambiguous final sends become needs_review rather than being blindly retried. Staff must check the destination before retrying uncertain outcomes.
- Published posts are not automatically duplicated by subsequent edits. Editing or deleting content on the website does not edit/delete already delivered external posts.

State group	Meaning
queued / preparing / publishing	Waiting, preparing media/caption, or final provider send
retry / failed / needs_auth	Retry scheduled, failure requiring action, or reconnect needed
needs_review	Delivery may have happened; inspect before retry
processing / inbox	TikTok transfer/status stages; not proof of public publication
published / cancelled	Receipt-backed completion or unsent job cancelled

Social photos are converted with Sharp into 1080 × 1350 JPEG snapshots, reused when possible and served from unguessable URLs for seven days. Expired social media is cleaned up. These files must remain reachable by providers.

The queue reduces duplication risk; it cannot guarantee exactly-once delivery across every third-party failure. A stopped host cannot publish on schedule.

Source: lib/social/queue.ts; lib/social/worker.ts; lib/social/media.ts; scripts/social-worker.ts; instrumentation.ts

WhatsApp subscriptions and translation

Two distinct WhatsApp uses

Ordering creates a prefilled wa.me link that the customer sends manually. Marketing is a separate Cloud API integration that sends approved templates only to explicitly subscribed recipients. Order and reservation contacts are not silently added to marketing lists.

- On /subscribe, the guest acknowledges consent and sends JOIN NVO or ABONNER NVO. Merely opening the page/chat does not create a subscription.
- Incoming webhooks must have a valid Meta signature and correct business phone ID. Message IDs are deduplicated.
- STOP unsubscribes, cancels unsent messages and is rechecked before each recipient send. Older delayed subscribe events cannot overwrite a newer STOP.
- Per-recipient records distinguish sent, delivered, read, failed, skipped and uncertain outcomes. A sender connection validates configured approved templates.

Local translation

The bilingual editor requests English/French translation through an authenticated API. A worker thread loads the two OPUS models using Transformers and ONNX WebAssembly CPU inference. Jobs are queued, input is chunked, results are cached by content hash in SQLite, and failed/timed-out requests show a retryable error.

Automatic platform captions primarily use a local rule-based editorial engine; they do not require a paid AI key. An optional server-side AI caption integration exists when explicitly configured. Translation and caption generation are separate systems.

Measured in the preceding local review: the model folder occupied about 271 MB and a standalone two-direction translation run peaked around 891 MB of process memory. This is a Windows test, not a production Linux sizing guarantee. Review translations before publishing.

Source: components/subscribe.tsx; lib/social/whatsapp.ts; lib/translation.ts; scripts/translation-worker.mjs; lib/social/captions.ts

Analytics, SEO and accessibility

Guest Activity

The dashboard reports consented visitor browsers, visits, page views, meal-unit additions/removals and action clicks. Visits use a 30-minute inactivity boundary; reports cover 30 days and group dates in Cotonou time. Browser identifiers are not verified people.

Collection excludes recognized staff browsers, authenticated staff, local development, sample-preview mode, non-public pages and recognized bots. The client respects declined consent and Do Not Track. Earlier mixed analytics remain version 1 and are separated from corrected totals. Unknown bots or changed cookies can still affect counts.

Search implementation

- Server-rendered English/French metadata, canonical URLs and language alternatives.
- Restaurant, menu, menu-item and breadcrumb structured data are based on the public catalogue; hidden prices and invented reviews are excluded.
- Sitemap uses eligible public entries and both languages; sample detail URLs are excluded. Sample-preview mode produces an empty sitemap and noindex metadata.
- Robots excludes admin, API, cart and referral paths. Robots directives are not access control; backend authorization remains required.
- Open Graph/Twitter imagery, favicon, image alt text, responsive images and local fonts support sharing and discoverability.

Accessibility and performance

The UI includes a skip link, labelled controls, responsive layouts and reduced-motion preferences. Existing browser suites run axe checks. The project records a prior local French-menu Lighthouse result of SEO 100, accessibility 100, best practices 100 and performance 80; those historical laboratory scores were not rerun for this report and do not guarantee live user performance or search rankings.

Source: lib/analytics.ts; components/guest-activity.tsx; lib/seo.ts; app/sitemap.ts; app/robots.ts; research/current-menu-and-seo.md

Backend security and data boundaries

Control	Implementation and practical limit
Staff authentication	Salted script password hashes; random session tokens stored as digests; HttpOnly cookies and 12-hour sessions
Authorization	Server-enforced role checks on privileged reads/writes; UI visibility alone is not trusted
Request origin	Mutating application and publishing routes compare Origin with Host
Input validation	Zod schemas, field lengths, quantity/price/date limits and request-size checks
Rate limiting	SQLite counters for login and sensitive operations; effectiveness depends on reliable identity signals
Coupon IP trust	Custom server signs socket-derived IP; explicitly trusted reverse proxy must overwrite the selected header
Social tokens	AES-256-GCM encryption with account-bound context; stable private encryption key required
OAuth	Expiring state bound to browser cookie and staff account; one-time consumption
Webhooks	Meta HMAC signature checks and inbound deduplication
Media paths	Restricted local paths, generated filenames, file-size and basic signature checks
Public catalogue	Filters drafts/archived content and suppresses hidden prices
Browser headers	nosniff, DENY framing, referrer and permissions policies

This is a source-level description, not an independent security certification. Launch work still includes staff recovery, abuse controls, retention policy, backups, TLS/proxy validation and live integration acceptance.

Private environment values, local admin credentials, database contents and uploaded customer-associated records must not be committed to a public repository. This report includes public business settings and schema names only; it omits passwords, encrypted tokens, session identifiers and customer rows.

Source: lib/auth.ts; lib/validation.ts; lib/network.ts; lib/social/security.ts; lib/social/accounts.ts; next.config.ts; .gitignore

Persistence, backup and recovery

SQLite uses WAL mode, foreign-key enforcement and a busy timeout. Content entries store their flexible fields as JSON while authentication, operations, coupon ownership, social jobs and audit trails have dedicated tables. Schema creation and incremental migrations run in `lib/db.ts`.

Location	Must persist / be recoverable
<code>data/nvo.sqlite</code>	Restaurant data, staff access, sessions, orders, coupons, publishing state and settings
<code>data/uploads/</code>	Photos uploaded through the admin workspace
<code>data/models/</code>	Local translation models; can be re-downloaded but needed for translation
<code>data/social-media/</code>	Temporary publicly accessible snapshots used by in-flight publishing jobs
Private environment / encryption key	Provider credentials and key needed to decrypt saved connections
<code>public/</code>	Versioned website images, videos and fonts
<code>.next/</code> and <code>node_modules/</code>	Build/runtime artifacts; regenerate on the target operating system

- Use SQLite's online backup API or a coordinated stopped-service backup. Copying only a live database file can miss WAL transactions.
- Back up uploaded files and private encryption configuration, with access controls appropriate to personal and credential data.
- Keep a copy away from the application server/account. A backup on the same disk does not protect against loss of that server.
- Restore to a separate environment and verify data, admin access, uploads, coupons and queue state before relying on the backup process.
- Avoid accidentally publishing restored jobs during recovery tests; keep the test worker off. Preserve delivery history and review uncertain jobs.

Production backup scheduling, external storage selection and a demonstrated restore are still deployment tasks. The Oracle template documentation describes them but has not installed them.

Source: `lib/db.ts`; `scripts/import-menu.ts`; `deploy/oracle/README.md`; `.gitignore`

Hosting and deployment plan

The selected direction in the project discussion is Oracle Cloud Always Free, subject to successful account access and server availability. No Oracle server or public deployment has been verified in this workspace.

Setting	Prepared direction
Server	Ubuntu ARM-compatible instance; proposed 2 OCPUs / 6 GB RAM / 50 GB boot disk, subject to console free eligibility
Runtime	Node 24; install dependencies and build on Linux rather than copying Windows binaries
Application	/srv/nvo; dedicated nvo service user; custom server on private port 3000
Public entry	Caddy HTTPS reverse proxy; temporary controlled hostname or final domain
Persistence	Preserve data directory and stable secret configuration across deployments
Client IP	Local reverse proxy overwrites X-Real-IP; application trusts only configured proxy addresses
Restart	systemd unit prepared for restart and boot startup
Backups / monitoring	Must be configured and validated before operational reliance

Files under `deploy/oracle/` include `nvo.service`, `Caddyfile`, `production.env.example` and the deployment guide. They are preparation templates, not evidence of installed or tested Linux services. The environment template uses `/etc/nvo/nvo.env` and binds the application to `loopback`.

The app does not require an expensive managed database at this size. It does require permanent disk and a process able to perform background work. Moving to serverless hosting would require changes to storage, background execution and the custom server's IP handling.

Oracle account/login issues are currently unresolved in the conversation. Free capacity, eligibility, idle reclamation and provider terms must be checked at provisioning time. This report does not promise uninterrupted free hosting or quote a live provider price.

Source: `deploy/oracle/*`; `server.mjs`; `README.md`; prior local resource review

Installation and daily operation

Existing local project

```
npm install
npm run setup
npm run setup:translation
npm run build
npm start
```

Node 24 or later is required. The production entry point is `server.mjs`. `npm run dev` uses the same custom server in development mode. Default local URLs are `http://localhost:3000` and `/admin`. Setup preserves an existing `.env.local`; private local credentials are in `private/LOCAL-ADMIN.txt` and are intentionally not reproduced here.

Operational commands

Command	Purpose
<code>npm run setup:social</code>	Generate missing encryption/webhook verification secrets without printing them
<code>npm run menu:import</code>	Preview current-menu import
<code>npm run menu:import -- --apply</code>	Apply through staff API with backup; preserve staff edits
<code>npm run showcase</code>	Install labelled sample editorial content once; use deliberately
<code>npm run social:worker</code>	Run external publishing worker against the same persistent database
<code>npm run typecheck</code>	TypeScript validation
<code>npm test</code>	Isolated automated business/integration unit tests
<code>npm run test:browser / test:design</code>	Browser workflows on a dedicated test database
<code>node scripts/audit.mjs</code>	Local Lighthouse audit against configured test URL

For updates: back up data, build/install in a controlled release, preserve the data directory and encryption key, restart the service, then verify homepage, menu, staff login, uploads, translation and queue heartbeat. Staff should manage restaurant content through admin rather than reseeding or replacing the database.

Source: `package.json`; `scripts/setup.mjs`; `scripts/setup-social.mjs`; `README.md`

Verification and remaining acceptance work

53 passed · 0 failed

npm test · 28 September 2026 · isolated data

The fresh run covered business rules, coupons, growth/analytics, menu and SEO, social publishing and social edge cases. Mock providers prevent real external posts or messages.

- Pricing: hidden totals, quantities, availability, duplicate lines and coupon ownership.
- Campaigns: capacity, identity/network checks, activation, expiry and counter redemption.
- Analytics: event deduplication, visit boundaries, staff exclusion, correct quantity deltas and time grouping.
- Menu: import idempotency, preservation of staff edits, bilingual routes, structured data and public visibility.
- Publishing: token encryption, OAuth replay protection, atomic saves, scheduling, independent failures, worker leases, uncertain sends and TikTok inbox handoff.
- WhatsApp: signature/consent checks, STOP ordering, receipt tracking, template validation and no automatic resends.

Evidence that predates this PDF

The project contains browser/design/accessibility outputs under artifacts/, plus documented build and Lighthouse results from earlier development. The prior session confirmed HTTP 200 for key local pages/APIs. Those are historical checks, not fresh public deployment or provider tests.

Required before launch

Validate the Linux build, ARM translation, HTTPS, correct proxy IP handling, login, database/upload persistence across restart, backups and restore, owner-approved content/policies, mobile WhatsApp routing, social account callbacks and one approved live publish per channel. Browser write tests must use a separate database.

This report generation did not rerun the production build or destructive browser suites, send messages, deploy the application or change restaurant content.

Source: tests/*.test.ts; artifacts/; research/current-menu-and-seo.md; README.md

Owner handover and outstanding scope

Information and approvals to collect

- Final business name, business email and kitchen/dining/delivery opening hours.
- Confirmed address/map entrance, working WhatsApp destination and final social profile ownership.
- Delivery areas/charges, menu portion differences, availability, reservations and cancellation policies.
- Approved testimonials, recognition permissions, genuine stories/events and offer/redemption terms.
- Domain ownership and a stable operator-controlled hosting account. NVO.com ownership/availability is not established by this application.
- For integrations: authorized Facebook Page, linked professional Instagram, TikTok developer setup and WhatsApp Cloud API sender/templates as applicable. Enter secrets privately, not in editorial fields.

Requested features not complete

Gap	Current boundary
Verified customer accounts / OTP	Browser-linked guest ownership only
Cross-device wallet	No account-based cross-device synchronization
Recurring randomized coupon drops	Manual/configurable campaign windows exist
Automatic referrer incentives	Referral visit/claim window exists; completed-sale reward automation absent
Advanced meal customization	Separate menu entries; no full modifiers/options engine
Staff account management and recovery	Backend roles exist; management/recovery UI incomplete
Abandoned coupon holds	Automated expiry/release policy still needed
Video, Reels, carousels and direct TikTok publishing	Current publishing flow is photo-based, with TikTok inbox completion
Production operations	Hosting, recovery, retention and live-provider acceptance remain

The next practical milestone is a stable review deployment with preserved data. Then complete business content and provider activation, verify real workflows, and only afterward advertise the application as operational.

Source: README.md; instructions.txt; research/social-publishing.md; research/owner-activation-guide.md

Public and session API reference

Method	Endpoint	Purpose
GET	/api/catalog	Filtered public settings and entries
GET	/api/offers/availability	Campaign remaining capacity and claimed state; verified IP
GET	/api/wallet	Current browser-owned public reward fields
GET	/api/admin/session	Staff session and bootstrap-configuration status
GET	/api/admin/data	Authorized staff dashboard; role-filtered data
POST	/api/admin/login	Credential validation and session creation
POST	/api/admin/logout	Invalidate session and cookie
POST	/api/analytics	Validated consented activity collection
POST	/api/quote	Server-authoritative cart/coupon quote
POST	/api/order	Idempotent saved order and WhatsApp URL
POST	/api/reservation	Saved reservation request and WhatsApp URL
POST	/api/claim	Named campaign/referral claim
POST	/api/reward/saved	Record owned card save request
POST	/api/reward/name	Name an eligible owned legacy reward
POST	/api/referral	Create/share browser-owned referral
POST	/api/referral/visit	Record invited visit for claim window

The main API uses a catch-all route and dispatches by path. JSON responses use no-store caching. Mutations validate Origin, request bodies and applicable roles. Webhooks instead use provider-specific verification. Consult the source schemas for exact payloads; these endpoints are not all anonymously writable.

Source: `app/api/[...path]/route.ts`; `app/api/admin/social/[action]/route.ts`; `app/api/social/*`; `app/uploads/[file]/route.ts`; `app/social-media/[id]/route.ts`

Staff and integration API reference

Method	Endpoint	Purpose
POST	/api/admin/upload	Authorized photo upload
POST	/api/admin/media/delete	Delete unreferenced uploaded photo
POST	/api/admin/settings	Owner-only restaurant settings
POST	/api/admin/entry	Content save plus atomic publishing plan/jobs
POST	/api/admin/delete	Archive entry and cancel unsent jobs
POST	/api/admin/order	Change operational order state
POST	/api/admin/quote	Staff quote for pending-price order
POST	/api/admin/reservation	Confirm/cancel reservation
POST	/api/admin/translate	Authenticated English/French translation
POST	/api/admin/coupon	Staff coupon lookup
POST	/api/admin/coupon-quote	Calculate counter-sale reward
POST	/api/admin/redeem	Validate payment and redeem at counter
GET	/api/admin/social/{dashboard,plan,audience,deliveries}	Publishing state, saved plan, subscribers, delivery results
POST	/api/admin/social/{captions,job,connect,select,account,unsubscribe}	Captions, job actions, account connection/defaults and opt-out
GET	/api/social/callback/{meta,tiktok}	OAuth callback
GET / POST	/api/social/whatsapp/webhook	Webhook verification / signed provider events
GET	/uploads/[file]; /social-media/[id]	Public media with path/expiry checks

The main API uses a catch-all route and dispatches by path. JSON responses use no-store caching. Mutations validate Origin, request bodies and applicable roles. Webhooks instead use provider-specific verification. Consult the source schemas for exact payloads; these endpoints are not all anonymously writable.

Source: `app/api/[...path]/route.ts`; `app/api/admin/social/[action]/route.ts`; `app/api/social/*`; `app/uploads/[file]/route.ts`; `app/social-media/[id]/route.ts`

Database schema reference 1

analytics

Versioned guest activity events

id TEXT · PK · visitor TEXT · event TEXT · page TEXT · source TEXT · created_at TEXT · version INTEGER · session_id TEXT · item_id TEXT · quantity INTEGER

analytics_excluded_visitors

Known internal/staff browser exclusions

visitor TEXT · PK · created_at TEXT

audit

Staff actions and affected entities

id TEXT · PK · staff_id TEXT · action TEXT · entity TEXT · created_at TEXT

counter_sales

Verified coupon sale receipt

id TEXT · PK · reward_id TEXT · staff_id TEXT · data TEXT · created_at TEXT

entries

Menu, editorial and campaign JSON records

id TEXT · PK · kind TEXT · data TEXT

media

Uploaded photo metadata

id TEXT · PK · name TEXT · url TEXT · created_at TEXT

migrations

Applied schema versions

version INTEGER · PK · applied_at TEXT

Column inventory is read from the current local SQLite schema. PK marks primary-key components. This section contains field names only, not personal data or secret values. JSON columns carry application-defined structured objects.

Source: data/nvo.sqlite PRAGMA table_info (read-only); lib/db.ts

Database schema reference 2

orders

Request data, state and unique retry key

id **TEXT** · **PK** · customer_id **TEXT** · data **TEXT** · status **TEXT** · created_at **TEXT** · request_key **TEXT**

rate_limits

Request counters and reset time

key **TEXT** · **PK** · count **INTEGER** · reset **INTEGER**

referral_visits

Per-invitee first opening time

referral_id **TEXT** · **PK** · customer_id **TEXT** · **PK** · opened_at **TEXT**

referrals

Referral owner and creation time

id **TEXT** · **PK** · customer_id **TEXT** · created_at **TEXT**

reservations

Table requests and staff state

id **TEXT** · **PK** · data **TEXT** · status **TEXT** · created_at **TEXT**

rewards

Issued ownership, terms, expiry, identity and redemption state

id **TEXT** · **PK** · code **TEXT** · customer_id **TEXT** · campaign_id **TEXT** · title **TEXT** · status **TEXT** · claimed_at **TEXT** · active_at **TEXT** · expires_at **TEXT** · terms **TEXT** · order_id **TEXT** · referral_id **TEXT** · claimant_name **TEXT** · network_hash **TEXT** · saved_at **TEXT**

security_config

Internal security material

key **TEXT** · **PK** · value **TEXT**

Column inventory is read from the current local SQLite schema. PK marks primary-key components. This section contains field names only, not personal data or secret values. JSON columns carry application-defined structured objects.

Source: data/nvo.sqlite PRAGMA table_info (read-only); lib/db.ts

Database schema reference 3

sessions

Hashed staff session tokens and expiry

token **TEXT** · PK · staff_id **TEXT** · expires **INTEGER**

settings

Singleton restaurant configuration

id **INTEGER** · PK · data **TEXT**

social_accounts

Platform account, encrypted token and defaults

id **TEXT** · PK · platform **TEXT** · remote_id **TEXT** · name **TEXT** · token **TEXT** · expires_at **INTEGER** · status **TEXT** · auto_publish **INTEGER** · created_at **TEXT** · updated_at **TEXT**

social_attempts

Job attempt outcomes

id **TEXT** · PK · job_id **TEXT** · outcome **TEXT** · message **TEXT** · created_at **TEXT**

social_caption_cache

Cached generated captions

id **TEXT** · PK · data **TEXT** · created_at **TEXT**

social_config

Provider configuration validated by integration

key **TEXT** · PK · data **TEXT**

social_jobs

Durable jobs, leases, payloads and provider receipts

id **TEXT** · PK · entry_id **TEXT** · account_id **TEXT** · revision **TEXT** · payload **TEXT** · status **TEXT** · attempts **INTEGER** · next_attempt **INTEGER** · lease_until **INTEGER** · lease_owner **TEXT** · container_id **TEXT** · media_url **TEXT** · provider_id **TEXT** · permalink **TEXT** · last_error **TEXT** · created_at **TEXT** · updated_at **TEXT**

Column inventory is read from the current local SQLite schema. PK marks primary-key components. This section contains field names only, not personal data or secret values. JSON columns carry application-defined structured objects.

Source: data/nvo.sqlite PRAGMA table_info (read-only); lib/db.ts

Database schema reference 4

social_media

Public photo snapshots and expiry

id TEXT · PK · file TEXT · expires_at INTEGER

social_media_cache

Source fingerprint to reusable snapshot

fingerprint TEXT · PK · media_id TEXT

social_oauth

Expiring browser-bound authorization state

state TEXT · PK · cookie_hash TEXT · staff_id TEXT · expires_at INTEGER · consumed INTEGER · candidates TEXT

social_plans

Per-entry destination/caption choices

entry_id TEXT · PK · data TEXT

social_worker

Worker heartbeat

id INTEGER · PK · heartbeat INTEGER

staff

Staff identity, password hash and role

id TEXT · PK · email TEXT · password TEXT · role TEXT

translation_cache

Content-hash translation cache

id TEXT · PK · text TEXT · created_at TEXT

Column inventory is read from the current local SQLite schema. PK marks primary-key components. This section contains field names only, not personal data or secret values. JSON columns carry application-defined structured objects.

Source: data/nvo.sqlite PRAGMA table_info (read-only); lib/db.ts

Database schema reference 5

whatsapp_deliveries

Per-recipient delivery ledger

id TEXT · PK · job_id TEXT · phone TEXT · status TEXT · provider_id TEXT · attempts INTEGER · next_attempt INTEGER · last_error TEXT · updated_at TEXT

whatsapp_inbound

Inbound message deduplication

id TEXT · PK · created_at TEXT

whatsapp_subscribers

Consent and opt-out state

phone TEXT · PK · name TEXT · status TEXT · consented_at TEXT · changed_at TEXT · source TEXT · last_timestamp INTEGER

Column inventory is read from the current local SQLite schema. PK marks primary-key components. This section contains field names only, not personal data or secret values. JSON columns carry application-defined structured objects.

Source: data/nvo.sqlite PRAGMA table_info (read-only); lib/db.ts

Environment and configuration reference

Group	Variable names	Purpose
Runtime	NODE_ENV; PORT; HOSTNAME_BIND; NVO_DB_PATH	Mode, socket and persistent database path
Public origin / cookies	SITE_URL; SECURE_COOKIES	Canonical HTTPS origin and secure cookie behavior
Owner bootstrap	ADMIN_EMAIL; ADMIN_PASSWORD	Initial account only; keep private
Proxy trust	TRUSTED_PROXY_IPS; TRUSTED_IP_HEADER	Exact proxy IP allowlist and overwritten client-IP header
Internal IP signing	NVO_INTERNAL_IP_KEY	Generated at server startup; not a public deployment secret
Maps / SEO	NEXT_PUBLIC_GOOGLE_MAPS_KEY ; GOOGLE_SITE_VERIFICATION	Optional restricted map key and domain verification
Analytics	ANALYTICS_ALLOW_LOCAL	Only for isolated test collection
Publishing	SOCIAL_TOKEN_KEY; SOCIAL_WORKER_MODE; SOCIAL_MEDIA_DIR	Encrypted credentials, worker mode and media storage
Meta	META_APP_ID; META_APP_SECRET; META_API_VERSION	Facebook/Instagram and shared Meta API setup
TikTok	TIKTOK_CLIENT_KEY; TIKTOK_CLIENT_SECRET	Authorization and upload integration
WhatsApp	WHATSAPP_ACCESS_TOKEN; WHATSAPP_PHONE_NUMBER_ID; WHATSAPP_BUSINESS_ACCOUNT_ID; WHATSAPP_WEBHOOK_VERIFY_TOKEN	Business sender authorization and webhook verification
Templates	WHATSAPP_TEMPLATE_EN/FR; WHATSAPP_TEMPLATE_EN_LOCALE /FR_LOCALE	Approved marketing-template names and languages
Optional caption AI	OPENAI_API_KEY; OPENAI_CAPTION_MODEL	Optional Responses-compatible caption generation

Only names and purposes are documented. Configure real values in private server environment settings. Do not put provider secrets in NEXT_PUBLIC variables. Preserve the encryption key during every deployment and restore.

Source: .env.example; lib/social/*; lib/seo.ts; server.mjs; deploy/oracle/production.env.example

Source map and handover references

Source	What to inspect or change
components/royal-home.tsx; shell.tsx	Homepage and global navigation/footer
components/menu.tsx; pages.tsx; meal-card.tsx	Menu, dish, cart, reservations and public content
components/provider.tsx; lib/cart.ts	Client data, language and persistent selection
components/admin.tsx; bilingual-fields.tsx	Staff editing and automatic translation UI
components/social-admin.tsx; subscribe.tsx	Publishing desk/composer and subscription UI
lib/db.ts; auth.ts; validation.ts	Persistence, access and validation
lib/catalog.ts; pricing.ts	Public visibility and authoritative totals
lib/rewards.ts; coupons.ts; network.ts	Claiming, redemption and IP verification
lib/analytics.ts; components/guest-activity.tsx	Activity collection and reporting
lib/social/*.ts	Connection, queue, provider, media and consent logic
lib/seo.ts; app/sitemap.ts; proxy.ts	Search metadata, sitemap and language URLs
lib/current-menu.ts; menu-import.ts; menu-photos.ts	Source menu, safe import and image provenance
research/social-publishing.md	Detailed provider activation and daily operation
research/owner-activation-guide.md	Staff handover and activation requirements
research/current-menu-and-seo.md	Menu transcription and historical SEO evidence
deploy/oracle/README.md	Prepared hosting plan and validation checklist

Primary requirements: instructions.txt. Current implementation summary: README.md. Supporting evidence: tests/ and artifacts/. Older research notes describe the state at their date; current source and verified local settings take precedence when they differ.

Report generated locally on 28 September 2026. To regenerate after changes: node scripts/generate-application-report.mjs. The verification paragraph records the test run performed for this edition; rerun tests and update that evidence before issuing a later edition.

Source: Repository source files and local records; no external account access performed